AI Math Tutor

Project Documentation

COLLEGE CODE : 0110

COLLEGE : Dharmamurthi Rao Bahadur Calavala Cunnan Chetty's Hindu College,

Team ID : SWTID-2026-9087
Team Size : 3
Team Leader : GOKUL K (9B19DFDA60F5004B3B8400C38B316F47)
Team member : ILAMUGILAN S (337B4B6F28A9D4196854947C0D34E710)

Team member : VIGNESHWARAN R (069481F9ABFC24985784C54A7CD841D0)

---

1. Introduction to the Project

Many students face difficulty when attempting to solve algebra problems from textbooks or handwritten notes. These challenges mainly arise due to problems in identifying mathematical symbols correctly, interpreting equations properly, and applying the correct sequence of solving steps. As a result, learners often make mistakes such as incorrect sign handling, wrong algebraic transformations, or simple calculation errors.

The **AI Mathematics Tutor** is developed to address these challenges by introducing an intelligent digital learning assistant capable of interpreting algebra problems directly from images. Students can capture or upload pictures of mathematical equations written on paper or printed in books. The system analyzes these images using **Optical Character Recognition (OCR)** together with advanced **Vision-Language Artificial Intelligence models** to identify mathematical symbols and convert them into well-structured **LaTeX mathematical expressions**.

Once the equation is converted into a machine-readable mathematical format, the platform applies **symbolic computation techniques** to solve the equation. The

system then generates a detailed, step-by-step explanation of the solving process so that students can clearly understand the logical reasoning behind each step.

In addition to solving the problem, the platform also examines the solving procedure to identify common algebra mistakes. It highlights possible error points and provides useful explanations so that learners not only obtain the correct answer but also improve their conceptual understanding. Through this interactive learning approach, the AI Mathematics Tutor transforms traditional problem solving into an intelligent and supportive digital tutoring experience.

2. Application Use Cases

Image-Based Equation Recognition

Students upload a photograph containing a handwritten or printed algebraic equation. The system processes the image and detects the mathematical symbols present in the equation. These detected symbols are then converted into a properly formatted **LaTeX mathematical representation**.

Outputs

- Recognized algebraic equation
- Structured LaTeX representation of the equation

Algebraic Equation Processing

After the equation is extracted from the image, it is interpreted using symbolic mathematics libraries. The system supports solving different types of algebraic equations, particularly **linear and quadratic equations**, and performs accurate mathematical computations.

Outputs

- Simplified algebraic expression
- Computed mathematical solution

Step-by-Step Solution Generation

To improve student understanding, the platform generates a logical sequence of steps explaining how the equation is solved. Each step is presented clearly to demonstrate the transformation of the equation until the final result is obtained.

Outputs

- Structured step-wise explanation
- Final computed answer

Error Detection and Educational Feedback

The system evaluates the solving process to identify frequent student errors such as incorrect sign handling, improper distribution of terms, or arithmetic miscalculations. When such issues are detected, the system provides explanations and helpful hints.

Outputs

- Error notifications and warnings
- Explanation of the identified mistake

3. System Architecture Overview

The **AI Mathematics Tutor** is designed as a modular intelligent system capable of interpreting algebra problems from images. The platform integrates technologies from artificial intelligence, computer vision, and symbolic mathematics to generate correct solutions along with detailed explanations.

The application interface is developed using the **Streamlit framework**, which enables an interactive and user-friendly environment where students can upload images and view results instantly.

The backend integrates **OCR technology, Vision-Language models, and symbolic mathematics libraries** to process and solve algebraic equations. The complete workflow consists of several stages including:

- Image preprocessing
- Symbol extraction

- Equation interpretation
- Mathematical solving
- Mistake detection
- Step-by-step explanation generation

By combining visual understanding with mathematical reasoning, the system ensures both **solution accuracy and conceptual clarity for students**.

4. Technology Stack

The system is built using multiple technologies that support different functionalities of the application.

Streamlit
A Python-based web framework used to build interactive user interfaces that allow students to upload images and view results dynamically.

Vision-Language Models (Gemini / OpenAI Vision)
Used to interpret extracted mathematical symbols and convert them into structured LaTeX equations.

OCR Engines (EasyOCR / Tesseract)
Responsible for extracting textual and symbolic information from uploaded images.

Python
The primary programming language used for implementing the backend logic and AI processing modules.

SymPy
A symbolic mathematics library used to simplify and solve algebraic expressions.

OpenCV
Used for image preprocessing operations such as resizing, noise removal, and contrast enhancement.

MathJax / LaTeX Rendering
Used to display mathematical expressions and step-wise solutions in a clear and readable format.

5. System Components

User Interface (Streamlit Frontend)

The frontend allows students to upload images of algebra problems and view the generated solutions. Mathematical equations and solutions are displayed using LaTeX formatting along with explanations and error notifications.

Image Processing Module

The uploaded images are preprocessed using OpenCV techniques such as resizing, noise reduction, and contrast adjustment to improve the accuracy of OCR recognition.

OCR Processing Layer

This layer extracts mathematical characters and symbols from the preprocessed images using OCR tools such as EasyOCR or Tesseract.

Vision AI Processing Layer

The recognized symbols are interpreted by the Vision-Language AI model and converted into structured LaTeX expressions while maintaining mathematical meaning.

Equation Solver Module

The solver uses the SymPy library to interpret LaTeX expressions, simplify algebraic equations, and compute the final solutions.

Mistake Detection Module

This module analyzes the solving process and detects common algebra mistakes such as incorrect sign usage, improper distribution of terms, or arithmetic calculation errors.

Solution Generation Engine

The system organizes the entire solving procedure into sequential logical steps and generates explanations for each stage.

Result Rendering Engine

The final results are displayed in the Streamlit interface using MathJax so that mathematical equations appear in a clear and readable format.

6. System Requirements

Software Requirements

- Python 3.9 or higher
- Streamlit Framework
- OpenCV Library
- EasyOCR or Tesseract OCR
- SymPy Library
- NumPy and Pillow libraries
- Gemini API or OpenAI Vision API access

Hardware Requirements

- Laptop or desktop computer
- Mobile phone or camera for capturing equation images
- Stable internet connection

7. Development Workflow

Phase 1 – AI Integration Setup

The first stage involves configuring the **Gemini AI environment** and preparing the system for image-based equation recognition.

Main tasks include:

- Creating an account in Google AI Studio
- Generating a secure API key
- Enabling multimodal processing for image and text inputs
- Testing connectivity using sample algebra images

API credentials are stored securely using environment variables, and prompts are designed to improve LaTeX conversion and mathematical reasoning accuracy.

Phase 2 – Core System Development

In this stage, the main application structure and the equation processing pipeline are implemented.

Key development tasks include:

- Designing the Streamlit application layout
- Implementing image upload and preview features
- Developing the image processing pipeline
- Integrating OCR extraction with LaTeX conversion
- Handling incomplete or incorrectly formatted equations

Phase 3 – Backend Implementation

The backend system responsible for solving equations and detecting mistakes is developed using Python.

Main functionalities include:

- Converting LaTeX equations into SymPy expressions
- Solving linear and quadratic equations
- Simplifying symbolic expressions
- Detecting common student mistakes
- Generating clear step-by-step explanations

Additional utility functions are also implemented to manage image validation, error handling, logging, and accuracy scoring.

Phase 4 – User Interface Development

The frontend is designed to be simple, interactive, and student-friendly.

Key interface features include:

- Homepage with project overview
- Image upload and preview functionality
- LaTeX equation display
- Solution generation interface
- Step-by-step explanation cards
- Mistake detection alerts and hints

The interface also includes loading indicators, responsive design for mobile devices, and clear user instructions.

8. Testing and Optimization

Comprehensive testing is conducted to ensure system reliability and accuracy.

Functional Testing

The application is tested with various algebra problems including both handwritten and printed equations.

Edge Case Testing

Testing also includes challenging scenarios such as:

- Blurry or low-quality images
- Incomplete equations
- Unsupported symbols

The system is designed to provide user-friendly feedback instead of crashing.

Quality Evaluation

System performance is evaluated using several criteria:

- Accuracy of final solutions
- Clarity of step-by-step explanations
- Correct LaTeX formatting
- Effectiveness of mistake detection

Performance Improvements

System efficiency is improved by:

- Optimizing AI prompts
- Reducing unnecessary API requests
- Improving image preprocessing techniques

9. Final Evaluation

Testing results demonstrate that the system performs accurately when processing clear algebra images. The generated step-by-step explanations are easy to understand, and the mistake detection module successfully identifies common algebra errors made by students.

The application maintains consistent performance and stable output formatting across different types of input images.

10. Conclusion

The **AI Mathematics Tutor** project demonstrates the effective application of generative artificial intelligence in the field of education. By combining image recognition technology, symbolic mathematical computation, and intelligent explanation generation, the system provides students with an advanced learning assistant for solving algebra problems.

The platform converts handwritten algebra equations into structured mathematical expressions, solves them automatically, and explains the reasoning process in a clear step-by-step manner. Additionally, the system identifies common mistakes and provides corrective guidance to strengthen students' understanding of algebra concepts.

The modular architecture built using **Streamlit and Gemini AI** ensures scalability, reliability, and efficient performance. In future developments, the system can be expanded to support additional mathematical domains such as calculus, geometry, and statistics, along with personalized learning analytics.

Overall, the AI Mathematics Tutor represents a significant step toward the development of intelligent tutoring systems that support independent learning and modernize digital education.